Modeling Human Activity States Using Hidden Markov Models

Tedla Tesfaye Godebo · African Leadership University · Formative 2 (individual) · 26 October 2026

1. Background and motivation

My use case is a phone-based **inactivity monitor** that's being used by two groups using the same model. Learners learning for hours at a desk (I am studying two degrees at the same time) sit for long periods or stand. When that stretch goes by, around 45 minutes, the app detects it and urges the person to go walk. The same model can be used to indicate an older person that has a relatively long period of time without any walking in the day, and that person lives independently. This may be an early indicator of a fall, illness or decreased mobility, notifying a family member on the app. It is not important whether any particular moment occurs, but the length of a state and what comes next. This fits nicely into a Hidden Markov Model: The transition matrix tells how long people typically remain in each activity and Viterbi decoding transforms the noisy windows from the sensors into a clean sequence of activity runs, whose lengths can be verified. In this report, this core model is built and tested for four states: still, standing, walking and jumping.

2. Data collection and preprocessing

I recorded 52 training files, 13 per activity, with one phone and one fixed setup. Each activity has 1 min 50 s – 1 min 55 s of usable data after cleaning, above the 1 min 30 s minimum.

Setting	**Value**
Phone / app	iPhone 13 Pro Max (iOS), Sensor Logger v1.68
Sensors	Accelerometer (m/s², gravity removed) and gyroscope (rad/s), x/y/z
Sampling rate	100 Hz for both sensors (one participant, one phone)
Phone position	Held flat, screen up, at waist level; flat on a surface for *still*
Recording length	10–12 s raw, 7.8–10 s after cleaning
Session A	23 Oct 2026, files _01–_04: outdoor, university campus, pavement, Ancona
Session B	25–26 Oct 2026, files _05–_13: walking outdoors at Globo Ancona (pavement); still and standing in an indoor corridor (ceramic floor); jumping in a building courtyard (concrete); all in Ancona

Data-collection videos (whole body, phone in hand, recorded during the _01 file of each activity). All four videos (still, standing, walking, jumping): Google Drive folder

Files. Every recording is stored as data/train/<activity>/<activity>_NN.csv, with the columns timestamp_ns, seconds_elapsed, acc_x, acc_y, acc_z, gyro_x, gyro_y, gyro_z, activity. Accelerometer and gyroscope samples were aligned by nearest timestamp (within 5 ms; every sample matched). The original app exports are kept in data/raw/.

Sampling rate. There was one phone used only, thus no need to reconcile two phones. The real intervals are still jittering, however: the median interval is 10.03ms (or 99.7Hz) and the maximum interval is 12ms. The FFT features assume evenly spaced samples and linear interpolation is done to every recording on the specific 100 Hz grid of its time-stamps. By performing the same step, a phone recording at either 50 Hz or 200 Hz would appear in the same grid.

Cleaning. The first and last second of each file have been cut, since starting and stopping the file by tapping the screen creates visible spikes. Files are later restricted to a 10 s time frame, so each training file is 5–10 s long.

Windowing. The signal is sampled at 100 Hz and divided into **2 s windows, each containing 200 samples, and a step size of 1 s (which results in 50 % overlap).** Both cadences are approximately 3 cadence (walking steps or jumps per second) and a 2 s window will contain at least 3 complete walking steps or full jumps. It also provides a resolution of 100 / 200 = 0.5 Hz in the FFT, which is sufficiently fine to separate those rhythms. A 0.5 s window would result in a 2 Hz resolution or less than half the cycles (2 Hz); a 4 s window would result in a resolution of 3-5 window per file and blur the activity switches. The 1 s step provides 1 HMM observation per second that leads to 369 observations (91-96 observations per activity) in total.

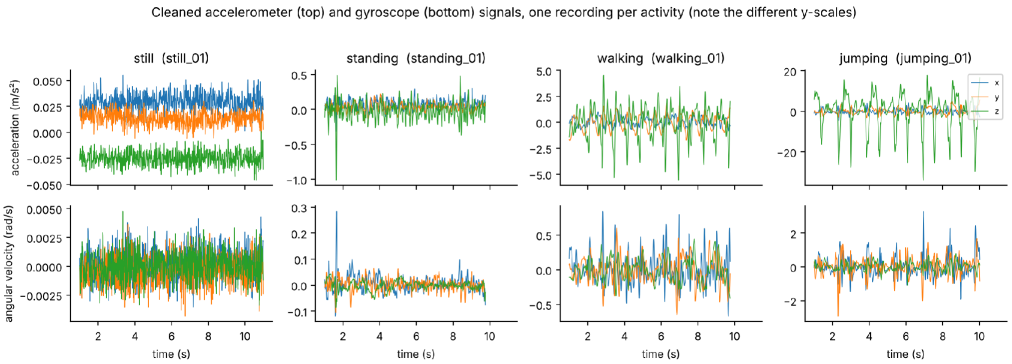


Figure 1. Cleaned accelerometer (top) and gyroscope (bottom) signals for one recording of each activity, generated by the notebook. Note the different y-scales: peak acceleration grows from about 0.05 m/s² (still) to about 35 m/s² (jumping).

3. Feature extraction and normalisation

The 10-number feature vector for each window is 6 time-domain features, and 4 frequency-domain features. The magnitude of the acceleration $|a| = \sqrt{(ax^2 + ay^2 + az^2)}$ and the magnitude of the gyroscope $|\omega|$ are computed and are not affected by the positioning of the phone in the hand. To minimize leakage at the window edges the FFT utilizes a Hann window on the mean-removed $|a|$.

Feature	Domain	Why it separates the activities
acc_mag_mean	Time	Average movement intensity: 0.03 m/s² still, 0.13 standing, 2.2 walking, 9.6 jumping
acc_mag_std	Time	Periodic impacts (steps, landings) make intensity vary strongly
acc_sma	Time	Signal magnitude area, a standard proxy for energy expenditure
acc_p2p	Time	Peak-to-peak range; only jumping produces landing spikes of 25–40 m/s²
gyro_mag_mean	Time	Rotation rate; separates phone on a table (none) from phone in a hand (small)
gyro_mag_std	Time	Swinging and bouncing of the phone while walking or jumping
acc_dom_freq	FFT	Strongest frequency in 0.5–20 Hz; a steady cadence for walking and jumping, random for noise
acc_spec_energy	FFT	Total spectral power $\Sigma\|X(f)\|^2/N$; differs by about 10^6 between still and jumping
acc_band_ratio	FFT	Share of power in the 0.5–3 Hz locomotion band: highest for walking, lower for jumping
acc_spec_entropy	FFT	Low for a clear rhythm (walking, jumping), near 1 for flat sensor noise (still, standing)

Normalisation. The features are in varying units and ranges and the features of the energy-type are over several orders of magnitude. The 7 magnitude and energy features that are strictly positive are first $\log_{10}$ transformed. Their differences in activities are multiplicative (each activity is about 10× more than the previous one) and the log puts them into evenly spaced Gaussian clusters, consistent with the Gaussian emission model. All 10 features are then **z-scored,** $z = (x - \mu)/\sigma$. The mean and standard deviation are calculated using the training windows only, and then used again without modification over the test data, which is why there is no leakage. Every feature is equally weighted in the Gaussian log-likelihood by performing z-scoring. No min–max scaling was considered as one jump spike at the end would crush all the other values.

4. HMM setup and implementation

The model is a 4-state Gaussian HMM written from scratch in numpy (no HMM library), with one state per activity.

Element	In this model
Hidden states Z	still, standing, walking, jumping
Observations X	One normalised 10-feature vector per 2 s window, one per second
Transitions A (4×4)	$P(z_t = j \mid z_{t-1} = i)$: will the activity continue or change in the next second?
Emissions B	$P(x_t \mid z_t = k)$ = Gaussian with mean μ_t and diagonal covariance σ^2_t
Initial distribution π	$P(z_1 = k)$, learned from the first window of each sequence

Because there are only ~90 training windows for each state, I used **diagonal** covariances. This data is too small to use to estimate 55 parameters in a full covariance matrix, which is what is required for the full covariance.If the covariance were full, it would require 55 parameters to be estimated from this data, which is too difficult and unreliable to do.

Algorithm choice. The activity is never observed, but only the sensor features are observed, so **Baum–Welch** (EM with the forward–backward algorithm) learns these parameters: π, A, μ and σ². **Viterbi** decodes the most probable sequence of

activities for a new recording. The confusion matrix, sensitivity and specificity and accuracy are those per window predictions. Viterbi suits these metrics because of the fact that there is a cost for switching between activities in the one window. A walking window inside a walking run is thus classified as walking, thereby lowering false positives (increasing the specificity), and increasing the overall **accuracy**, over the individual classification of each window.

Implementation details.

- Forward, backward and Viterbi run in log space with a custom logsumexp, which means that long sequences will not underflow.
- Using all 52 recordings, each recorded separately in a different window (6, 9 windows respectively), Baum–Welch trains on all the windows together.
- The program will terminate when the training objective changes by less than $\varepsilon = 10^{-4}$ between iterations. The maximum of 500 iterations is a safeguard and was never accomplished.
- For the still Gaussian, a variance floor of 10^{-3} avoids the still Gaussian from collapsing, because its windows are almost constant; π is smoothed with add-one smoothing.

Transition prior. In each training file, there is only one single activity, therefore the training data will only ever report "stay in the same state". The maximal likelihood would be plain, and Viterbi would never change activity in a test file. I add a few pseudo counts to the transition update (MAP-EM) so that I can have a few Dirichlet pseudo counts. They represent that transitions from neighbouring activities (still↔standing, standing↔walking, walking↔jumping) are more likely than transitions from distant activities (2 steps apart, still↔jumping). Each state observes approximately 70 self-transitions, so the data still determine each activity's duration.

Initialisation and convergence. The main model is started from the mean and variance of each of the labelled windows, and the main model converged in 3 iterations, since this is already close to the optimum start. I then tried to test the implementation harder by running the iteration of Baum–Welch **without any labels**, 10 times from 10 random starts, with each one taking 5-30 iterations. The top 3 runs all achieved the same solution and it correctly identifies 100 % of the training windows after matching states to activities. The other runs came to a halt at inferior local optima, whereas Baum–Welch was able to find the four activities.

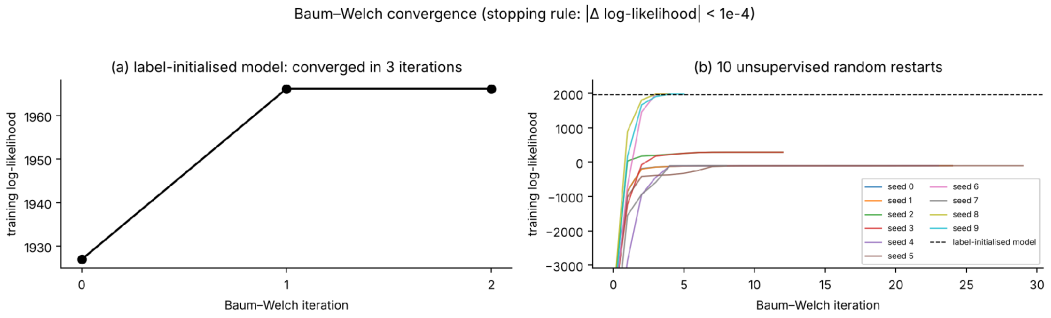


Figure 2. Training log-likelihood per Baum–Welch iteration. (a) Label-initialised model, stopped after 3 iterations. (b) 10 unsupervised random restarts; the best runs reach the same optimum as (a), the others stop at worse local optima.

5. Results and interpretation

The final model achieves a window **accuracy of 91.2 %** on the two hidden test recordings and all errors are between activities.

Learned parameters. The transition matrix (Figure 4a) remains unchanged with a probability of 0.92-0.95 per second. This translates to an approximate standing and walking time of 12 s and still and jumping of 20 s. The odds of transitioning to a neighbouring activity when still↔jumping (0.003) are about 10 times smaller. For each intensity feature, the emission means (Figure 3) increase monotonically from still to jumping. In the same way, the movements become more rhythmic and this corresponds to the order of the spectral entropy. The most spread is found for the still case, with a dominant frequency, where the peak is just random noise ($\sigma = 1.3$ in z-units).

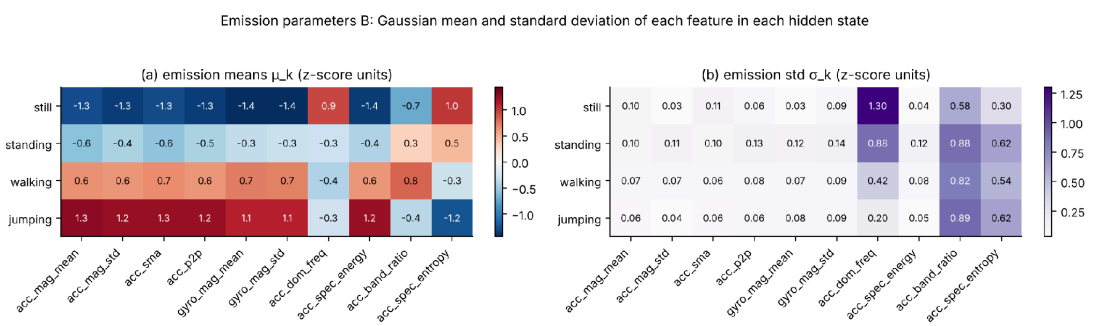
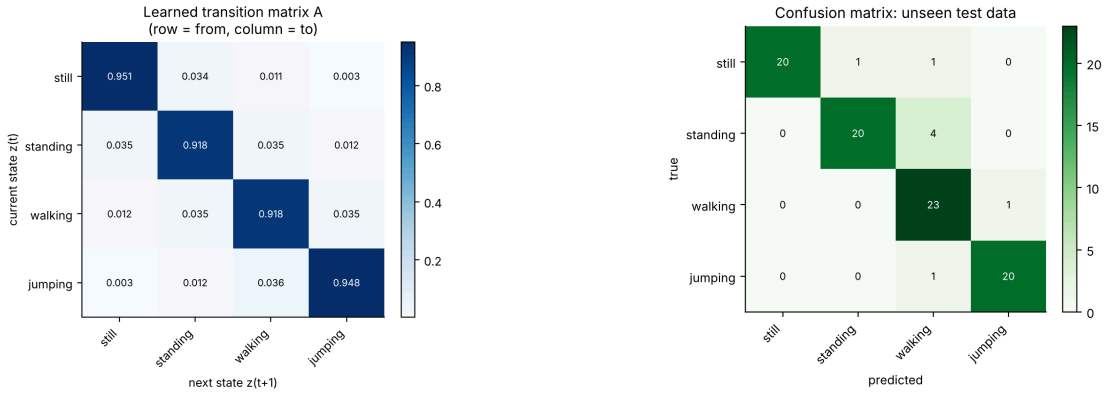


Figure 3. Emission probabilities B: (a) Gaussian mean and (b) standard deviation of each normalised feature in each hidden state.

Unseen test data. I recorded two new files on 26 Oct 2026, in a session and place not used for training. Each follows a fixed script of about 12 s per activity, recorded without stopping: test_01 still → standing → walking → jumping (48 s), and test_02 walking → standing → jumping → still (51 s). The switch points were confirmed on the raw signal, where every change is a clear step in intensity. The short moments of handling the phone between activities fit no class, so they are excluded from scoring: picking it up from the table in test_01 (12.9–14.3 s) and putting it down in test_02 (38.2–39.0 s). Neither file, nor the normalisation statistics, was used in training.

State (Activity)	Number of Samples	Sensitivity	Specificity	Overall Accuracy
Still	22	0.909	1.000	0.912
Standing	24	0.833	0.985	0.912
Walking	24	0.958	0.910	0.912
Jumping	21	0.952	0.986	0.912

Samples are 2 s windows (91 in total). test_01 scored 90.9 % and test_02 91.5 %.



(a) Learned transition matrix A *(b) Confusion matrix, unseen test data*

Figure 4. (a) Transition matrix (row = current state, column = next state, one step = 1 s). (b) Confusion matrix on the two test files (91 windows; rows = true, columns = predicted).

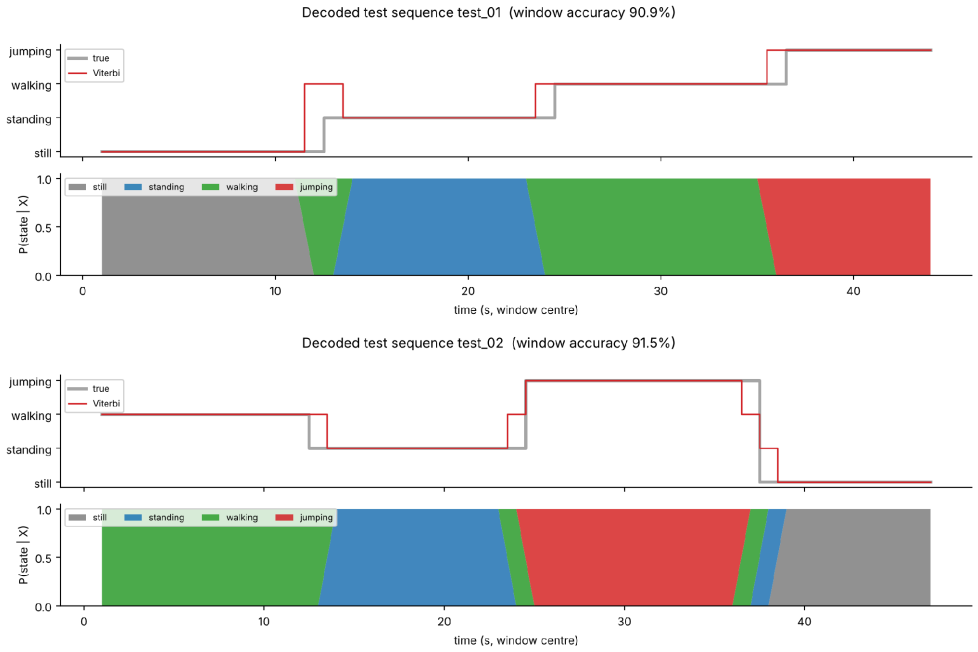


Figure 5. Viterbi-decoded activity sequence (red) against the true labels (grey) for test_01 (top) and test_02 (bottom), with the posterior state probabilities below each. Errors appear only at the switches.

Does the model generalise? Yes. All 8 errors are within 1 or 2 windows of a switch for the sequences in both files, and no sequence is coded twice. None in the middle of an activity. A very thorough test was conducted: I re-trained the entire pipeline using one recording session and tested it with the other, and it correctly labelled 369 windows, despite the two recording sessions taking place in different locations and on different floor surfaces. The difference between training and test accuracy (100 % - 91.2 %) is explained by the lack of transition files in the training files.

6. Discussion and conclusion

Easiest and hardest activities. Still was the easiest one with No False Positive (specificity 1.000). A hand-held phone rotates 15 times more quickly than a phone on a table, $|\omega| \sim 0.03$ rad/s, compared to $|\omega| \sim 0.002$ rad/s. Landing spikes were also easy to jump on, as they are the only ones that have a 25–40 m/s² landing rate. It was the most difficult (sensitivity 0.833) to stand. All four of its errors were windows near a walking segment that was called walking: the first or last steps in the 2 s window were still in the segment. Walking is thus the most general activity, having the lowest specificity (0.910).

Do the transition probabilities reflect real behaviour? Partly. High self-transitions (expected runs of 12-20 s) and ranking of switches are realistic: people do not go from still to jumping, rather they go from sitting to standing to walking. However, most of the off-diagonal values are from the previous I which I had already fixed; since there are no transitions at all in my training files. This matrix represents a much more dynamic reality of real daily behaviour: a student sits still for 20 s, not 20 min. In the case of the inactivity monitor, A should be re-estimated from long and continuous recordings of real days.

Effect of sensor noise and sampling rate. For 100 Hz there was sufficient sampling rate; the useful motion was below 20 Hz and the jitter in the timestamps (at most 12 ms) were eliminated by resampling. The most important consideration for the quiet classes was the noise. The log transform and the Gyroscope feature were critical because the signals are very similar in both configurations, except for low amplitude levels near the sensor's noise floor, where there is some tremor-level difference. Picking up the phone between activities (not one of the four mentioned classes) was the largest "noise": This was not one of the four classes and the model classified it as walking or standing.

Improvements.

- Capture training data with **transitions** – multi-activity training sessions.
- Increase the number of **transition/handling states** or use shorter windows at switches to minimize boundary errors.
- Gather data from a larger sample, bag orientation (pocket, bag) as well as phone orientation, and the model will also perform if the orientation and sampling rate is different.
- Once you have more data, try full-covariance emissions or Gaussian mixtures to get the gravity/orientation channel you need to separate sitting, standing and lying, which the inactivity monitor needs.

Conclusion. A from scratch Gaussian HMM with 10 time- and frequency-domain features was able to recognize 4 activities from the phone motion data. It achieved 100 % on held-out recording sessions and 91.2 % on two unseen continuous test recordings, with the only errors being made at activity switches. Even when labels were not provided, Baum–Welch converged reliably to the four activities. The clean activity runs yielded by Viterbi decoding were just what an inactivity monitor requires to determine the length of time someone remained inactive. The largest restriction is the need to have realistic recordings that are continuous over time for the benefits of the duration-based alerts to be taken for granted.

Repository: https://github.com/Teddy-2004/hmm-activity-recognition.git